

Banking Shield: Machine-Checked Absence Claims for Privacy-Sensitive AI Explanations

Mohammad Raouf Abedini 

Department of Computing

Macquarie University

Sydney, Australia

mohammadraouf.abedini@students.mq.edu.au

<https://raoufabedini.dev>

Abstract—AI-style explanations are attractive in sensitive research prototypes because they can help participants understand what a system did. They are also the place where privacy and capability boundaries most often blur: an explanation layer may receive data the prototype should not hold, send that data to a provider, or describe capabilities the prototype does not have. Banking Shield tests a narrower design goal: can a prototype make useful explanation artifacts while producing machine-checkable evidence about what the explanation layer did not receive, did not transmit, and did not claim? We present a fictional, non-bank, research-only banking-adjacent prototype whose main artifact is bounded evidence of absence: gate-backed evidence that sensitive values were not recorded in the frozen evidence pack and did not enter the explanation payload. The system combines a fail-closed metadata firewall whose rejections become audit evidence, per-session tamper-evident HMAC audit chains with withdrawal semantics, and a deterministic offline AI-style privacy firewall: allowlist input construction, a deterministic mock narrative provider, a negation-aware output claim firewall, per-response privacy receipts, and a static source gate over the four AI-firewall modules for network primitives. At the evidence freeze, all automated gates passed (417/417 unit tests; 43/43 end-to-end checks; 27/27 security checks; three privacy audits), and five trusted internal testers completed 30 formative sessions with zero sensitive values in the evidence and 5/5 checklist comprehension of the system’s non-claims. The prototype does not evaluate banking effectiveness or live LLM safety. Its contribution is a reproducible pattern for turning privacy and overclaim boundaries into testable system properties.

Index Terms—data privacy, data minimisation, audit trails, message authentication, LLM guardrails, explainability, privacy by design

I. INTRODUCTION

Banking Shield is a fictional banking-adjacent research prototype. It has no connection to any bank, processes no real financial data, and makes no fraud-detection, scam-prevention, payment-safety, payee-verification, financial-advice,

Fictional, non-bank, research-only prototype. Not a fraud-detection, scam-prevention, payment-safety, payee-verification, financial-advice, compliance, or production-readiness claim. Author-prepared preprint; not yet peer reviewed. Source: <https://github.com/Raouf128/Project-Simurgh>. Evidence pack frozen at commit 92dabb4; claims re-audited at 3dcf21b. Substantial LLM assistance was used in drafting (Sec. XI).

or compliance claims. That constraint is not a footnote; it is the object of study. The prototype asks how a system can help a participant inspect a sensitive workflow while proving, within a bounded artifact set, that it did not collect prohibited values or let its explanation layer exceed the system’s actual capability.

Sensitive-domain prototypes usually argue safety through policy language (“we do not collect X”) and argue explanation quality through behavioural language (“the model rarely says Y”). Those arguments leave three engineering gaps:

- **G1: Explanation layers leak by default.** An LLM-backed explanation feature typically receives the session record and emits free text; nothing structural prevents sensitive input or unbounded output.
- **G2: Overclaims are unpoliced.** Nothing stops an explanation layer from asserting capabilities (“fraud detected”) that the system does not have; in regulated-adjacent domains this is the costliest failure mode.
- **G3: Absence of data is rarely evidenced.** Systems log what they did, not what they refused; a participant cannot verify that nothing sensitive was recorded.

Our thesis: *privacy-sensitive explanation should start with structure, not moderation*. A prototype should construct the smallest possible explanation payload, fail closed before provider invocation, block unsupported claims after generation, and leave receipts that let a participant or reviewer inspect the boundary.

Contributions:

- **C1** A metadata-only integrity-evidence architecture: a recursive forbidden-field firewall (46 key names, including prototype-pollution keys) whose rejections are audit events that escalate subsequent policy scoring; refusals become evidence (Sec. IV-A).
- **C2** A deterministic, offline AI privacy firewall: allowlist input contract, deterministic narrative provider, fail-closed output claim firewall, per-response privacy receipt, and a static source no-egress check over the four AI-firewall modules with a negative self-test (Sec. IV-B, Fig. 3).

- **C3** Claim discipline as a mechanism: a negation-aware scanner that blocks affirmative capability phrasing while passing required disclaimers, enforced at runtime and mirrored by repository CI (Sec. IV-C).
- **C4** A verifiable participation lifecycle: consent, withdrawal, HMAC audit chains, and aggregate-only human evidence, exercised in a formative internal dry run (Sec. IV-D, Sec. VI-B).

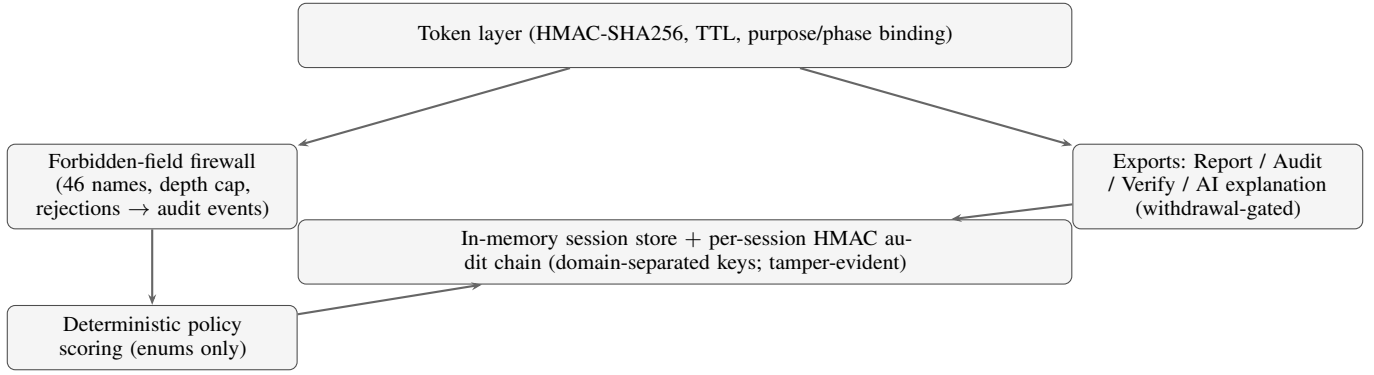


Fig. 1. Banking Shield architecture. Every write passes the forbidden-field firewall before validation; every export is token-bound and chained. The system is metadata-only by construction (C1).

II. BACKGROUND AND MOTIVATION

A Banking Shield session asks a participant to consent, complete one of five fictional scenarios, and inspect exports: a Report, an Audit listing, a Verify check, and, behind a default-off flag, an AI-style explanation (Fig. 2). The scenarios resemble sensitive banking-adjacent moments: a mock data-sharing consent screen, a mock payee-name check, a remote-access caution, a payment-pause prompt, and an AI-agent finance approval. All scoring is a local deterministic function over enumerated metadata. The labels safe/warning/critical are fictional prototype outputs, not calibrated alerts.

Three observations from the project evidence shaped the design:

- **O1.** Every prohibited input class the project tested (46 field names: credentials, OTPs, account identifiers, amounts, payees, device telemetry, and structural pollution keys) appeared only in attack-style tests. Each was rejected and logged. Refusal-as-evidence is therefore mechanically achievable within this prototype.
- **O2.** In the formative dry run, all five testers understood the non-claims (5/5 on fictional-only, no-bank-connection, no-fraud-detection, no-financial-advice, withdrawal), but only one of five initially understood the Report/Audit/Verify exports. Interpretability, not privacy, was the weak point. That result motivated the explanation layer and matches the broader finding that warning comprehension depends strongly on text characteristics and terminology [1], [2].
- **O3.** The explanation layer is the riskiest place to improve interpretability because it sits at the boundary between internal evidence and reader-facing language. The design therefore treats the explanation layer as an adversarial surface, not a cosmetic feature.

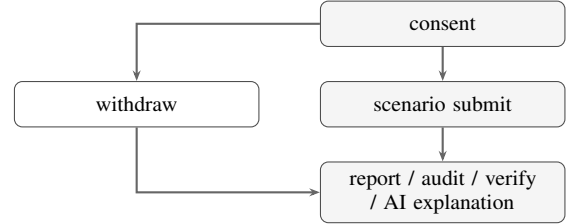


Fig. 2. Session lifecycle. Withdrawal blocks report and explanation export (403) but preserves audit/verify reads for transparency (C4).

TABLE I
ALLOWED VS FORBIDDEN DATA (T1).

Allowed into the narrative pay-load (allowlist)	Forbidden everywhere (46-name denylist, classes)
Hashed session id	Credentials and one-time codes
Scenario type (enum)	Account/card/payment identifiers
User action/decision category (enum)	Amounts, balances, payees, references
Policy outcome + score (deterministic)	Transaction/statement content
Manual-review flag (boolean)	Screens, recordings, keystrokes, clipboard
Privacy-assertion booleans	Window titles, process/app names, device ids
No free-text payload	__proto__, prototype, constructor

III. THREAT MODEL

Assets. A1 sensitive banking payloads (must never enter); A2 audit-chain integrity; A3 the claim boundary; A4 participant anonymity; A5 official policy-result integrity.

Trust assumptions and residual risks. The server, its secrets, and transport security are trusted; Phase B testers are trusted insiders. The audit chain is tamper-evident, not tamper-proof: the server holds the keys, a limitation shared by server-side secure-logging schemes generally, which motivates third-party or distributed verification in that literature [3], [4]. Denylist scanning is incomplete by construction. Receipts attest process, not ground truth. Withdrawal does not erase the metadata-only audit chain; the prototype discloses that choice. The evaluation

TABLE II
THREATS AND CONTROLS (T5).

Threat	Controls	Asset
T1 Sensitive/polluted submission	Recursive denylist firewall, depth cap, body cap, rejection audit events, score escalation	A1
T2 Provider exfiltration (future live LLM)	Allowlist input (hashed id, enums), 4 KB cap, static no-egress gate, deterministic mock provider	A1
T3 Capability overclaim in narrative	Output schema allowlist, length caps, negation-aware claim scan, fail-closed rejection	A3
T4 Narrative drifts official result	Equality check against the authoritative record	A5
T5 Audit-log tampering	Per-session HMAC chains, domain-separated keys; tampered fixtures fail verification	A2
T6 Token theft/replay	HMAC tokens, constant-time compare, TTL, purpose/phase binding, path/token match	A1, A4
T7 Withdrawal bypass	Withdrawn $\rightarrow$ 403 on report/explanation; audit/verify intentionally readable	A4

does not cover DoS beyond rate limiting, side channels, browser compromise, or a malicious operator.

IV. DESIGN

Banking Shield uses the same rule at every boundary: construct what is allowed, reject what is not, and record the rejection without recording the prohibited value. The architecture applies that rule to submitted metadata, exports, explanation inputs, explanation outputs, and participant withdrawal.

A. Metadata-only integrity evidence (C1)

Every write passes through a recursive forbidden-field firewall before schema validation. The firewall blocks 46 key names spanning credentials, financial identifiers, content fields, device telemetry, and the structural pollution keys `__proto__`/`prototype`/`constructor`; it also enforces a recursion-depth cap and a 16KB body limit. A rejection appends an audit event containing the route, reason, and field name, never the value. The session’s later policy score escalates after a rejection, so an attack attempt becomes part of the integrity record rather than a discarded exception. This operationalises a data-minimisation design principle—in the limited engineering sense of constructing only the metadata the stated prototype purpose requires—as an enforced, evidence-producing mechanism rather than a policy statement; we claim no regulatory compliance. The broader GDPR data-protection-by-design context that motivates this posture is analysed in the governance literature [5], [6]. *Alternative considered:* schema-level allowlisting alone. The project rejected that as insufficient: an allowlist validates shape, but it does not produce evidence of hostile attempts. Banking Shield therefore uses both controls. The allowlist defines the valid shape; the denylist converts prohibited attempts into auditable signal.

B. The deterministic AI privacy firewall (C2)

The explanation endpoint exists to make the report understandable without turning the explanation layer into a privacy sink. We use “AI-style” to denote the explanation-interface contract and guardrail pathway, not a live generative model evaluated in this study. It therefore has four fail-closed stages:

- 1) **Input firewall.** The narrative payload is *constructed*, not passed through: a hashed session id, scenario/action enums, the policy outcome, and the privacy-assertion booleans. The assembled payload is re-scanned against

the forbidden-field denylist (defence in depth) and capped at 4 KB. Failure returns a receipt, never a narrative.

- 2) **Deterministic provider.** Narrative text is an enum-to-template pure function: no randomness, no clock, no I/O. A static source gate checks that the four AI-firewall modules import no network primitive, and a negative self-test confirms the gate fails when a network primitive is injected. This is not a host-level egress-control claim.
- 3) **Output claim firewall.** The narrative must match an exact top-level schema (field allowlist), respect 600-character caps per field, contain a non-empty non-claims list, and pass a negation-aware scan over 29 affirmative-capability phrasings; an equality check proves the narrative did not drift the official result.
- 4) **Privacy receipt.** Every response, whether successful, blocked, or disabled, carries a receipt: `provider identity (deterministic_mock)`, `sensitive_payload_sent_to_ai:false`, `network_egress_used:false`, official-result and claim-guard flags, and a success-only SHA-256 narrative hash that is also written into the session’s audit chain.

Alternative considered: a live LLM behind the same firewall. The project rejected it for this paper because it would convert the no-egress property from static source evidence into an operational assertion. The input allowlist, byte cap, output schema, claim scan, and receipt mechanism would survive a future live provider. Determinism and reproducible narrative hashing would not; a live provider would require replay logging and separate provider-risk evaluation. The mock therefore establishes the contract and enforcement points, not live-model safety.

C. Claim discipline as a mechanism (C3)

The output firewall’s scanner is negation-aware. Required disclaimers such as “not fraud detection” and “not a fraud detection tool” pass; affirmative phrasing and weakened negation such as “not really a ...” are blocked. The scanner is intentionally conservative: a false positive blocks a narrative, and the paper claims no complete false-negative class. Repository CI enforces the same boundary with an overclaim-wording scanner. During integration, that scanner flagged the firewall’s own denylist because the denylist contains the phrases it blocks. The project resolved the conflict with an explicit scanner exclusion

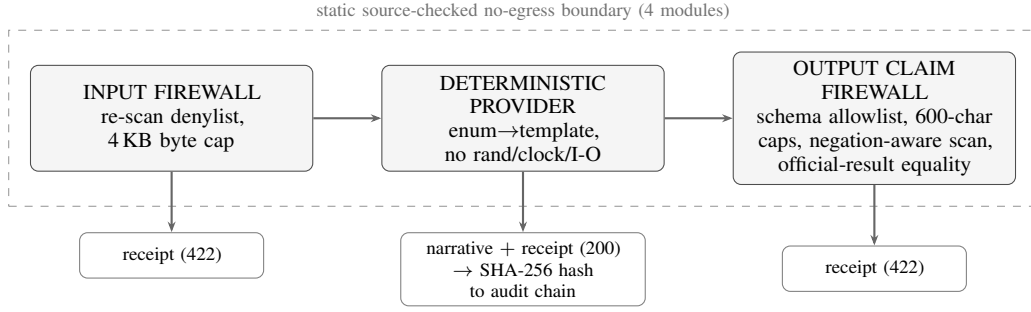


Fig. 3. The AI privacy firewall. The no-egress boundary is source-checked over the four AI-firewall modules; every stage fails closed to a receipt, never to a narrative (C2).

TABLE III
CLAIMS VS NON-CLAIMS (T4).

The paper claims	The paper does NOT claim
Metadata-only integrity evidence (about the process)	Fraud detection or scam prevention
A fictional banking-adjacent prototype	Real banking protection or payment safety
A trusted internal dry run (formative)	Real payee verification or financial advice
A deterministic AI privacy firewall (contract + gates)	CDR/APRA/AML/CTF compliance
AI-style explanation without sensitive payloads	Production readiness or bank-grade security

TABLE IV
GATE RESULTS AT EVIDENCE FREEZE (T3).

Gate	Result
Unit tests	417/417 pass
Banking smoke	14/14 pass
AI firewall smoke	5/5 pass
Full banking E2E smoke	43/43 pass (incl. tamper-evidence check)
Banking security audit	27/27 pass
Banking privacy audits	all 3 PASS
No-egress static gate	PASS (with negative self-test)
Dependency audit	0 vulnerabilities
Repository CI quality gate	green

rather than string obfuscation. Claim discipline is therefore checked at runtime and again at repository level.

D. Verifiable participation lifecycle (C4)

Each session carries an HMAC-SHA256 audit chain with domain-separated keys: participant-code hashing and chain signing cannot be cross-validated. Participants can export a Report, an Audit listing, and a Verify check of chain consistency. The Report contains explicit privacy assertions; the sensitive-data assertions remain `false` when the firewall has blocked prohibited input. Withdrawal blocks Report and explanation export, but keeps Audit and Verify readable so a participant can still inspect the chain. The human-evidence pipeline stores only aggregate counts.

V. IMPLEMENTATION

The implementation uses Node.js/Express, static HTML/CSS/JS tester pages, an in-memory session store, and shell/Node CI gates. The explanation panel writes through `textContent` only. The explanation endpoint sits behind a default-off flag requiring the exact string `"true"`; requests must be token-bound, path-matched, and within the read-rate limit. The prototype is single-node and research-only.

VI. EVALUATION

The evaluation is bounded by the prototype’s research-only scope. It asks three questions:

- **E1.** Do the structural controls pass their automated gates?

- **E2.** Can trusted insiders complete the lifecycle and understand the prototype’s non-claims?
- **E3.** Does the explanation firewall pass a clean narrative and block a poisoned one?

A. Gate results (E1)

Hypothesis: the prototype’s claimed structural properties can be checked mechanically. *Result:* at the evidence freeze, every automated gate passed (Table IV). The claim is deliberately narrow: the gates passed for the frozen prototype and evidence pack. The result does not establish production security or regulatory compliance.

B. Formative internal dry run (E2, exploratory)

Hypothesis: trusted insiders can complete the lifecycle and understand the non-claim checklist. *Result:* five trusted internal testers completed 30 sessions (Table V). No tester entered real banking values; the evidence pack contained zero sensitive values; the deterministic policy pattern reproduced identically across testers (safe/warning/warning/warning/safe across the five scenario types); all five withdrawals blocked later report export; and all five testers passed the five-item non-claim checklist. The weak point was export interpretability: only 1/5 initially understood the Report/Audit/Verify exports. A focused 3-session rerun confirmed revised copy, but it cannot establish improvement; we report it only as a design iteration. This is a formative dry run with $n = 5$ trusted insiders, not a user study and not a representative banking-customer sample.

TABLE V
PHASE B AGGREGATES (T2, EXPLORATORY).

Measure	Value
Trusted internal testers	5
Total sessions	30
Submitted scenario sessions	25 (5 per scenario)
Withdrawal sessions	5
Withdrawals blocking later report export	5/5
Real banking values entered	0
Sensitive values found in evidence	0
Forbidden payload structures in evidence	0
Comprehension: fictional-only	5/5
Comprehension: no bank connection	5/5
Comprehension: no fraud detection	5/5
Comprehension: no financial advice	5/5
Comprehension: withdrawal	5/5
Export interpretability (pre copy-patch)	1/5

C. Fixture studies (E3)

Hypothesis: the output firewall passes a clean narrative and blocks a poisoned one. *Result:* the accepted-explanation fixture, a clean generated narrative with receipt, passes all gates and contains no attack values. The rejected-claim fixture, a deliberately poisoned narrative asserting a detection capability, is blocked by the claim guard with the failed gate recorded in its receipt. The fixture pair demonstrates the firewall’s two required behaviours under controlled conditions.

VII. LIMITATIONS

The strongest limitation is also the point of the paper: the provider is a deterministic mock. No live LLM has been filtered, evaluated, or validated. The mock lets the paper isolate the contract around an explanation provider, but it does not measure real model behaviour.

The human evidence is formative. Five trusted insiders completed the dry run; the paper makes no statistical claim, no representative-user claim, and no banking-customer comprehension claim. The result supports lifecycle feasibility and checklist comprehension within a trusted internal setting.

The security evidence is bounded. Denylist scanning is incomplete by construction, and adversarial phrasing can evade a phrase scanner. The allowlist construction is the stronger control. The gates are written by the project they validate; negative self-tests and public reproducibility reduce that risk but do not remove it. The prototype is single-node and in-memory. The audit chain is server-keyed, tamper-evident to participants, not tamper-proof against the operator. The no-egress claim is a static source check over the four AI-firewall modules, not a network sandbox or host-level egress-control guarantee.

VIII. REPRODUCIBILITY AND REVIEW-SUBSTITUTE PACK

The artifact includes the paper, evidence pack, final audit, claim audit, and Stage B5-R review-substitute pack under `Papers/banking-shield/` and `docs/research/banking-pilot/`. The Stage B5-R pack simulates five external-style hostile reviews

(privacy/security, banking governance, HCI, AI safety, and reject-oriented reviewer #2 attack) and records author responses before preprint submission. This is not a replacement for formal peer review; it is a structured preprint-risk-reduction step. The core reproduction gates are:

```
npm test
bash scripts/smoke-banking-pilot.sh
SIMURGH_BANKING_PILOT_AI_EXPLAIN=true \
  bash scripts/smoke-banking-pilot-ai-firewall.sh
bash scripts/smoke-banking-pilot-full-e2e.sh
bash scripts/security-audit-banking-pilot.sh
node scripts/privacy-audit-banking-pilot.mjs
node scripts/privacy-audit-banking-pilot-phase-b.mjs
node scripts/privacy-audit-banking-pilot-ai-firewall.mjs
npm audit --audit-level=moderate
```

These gates support the paper’s mechanism and evidence-pack claims. They do not establish production security, regulatory compliance, live-LLM safety, or real-world banking effectiveness.

IX. RELATED WORK

Tamper-evident logging. Secure-logging schemes protect log integrity and authenticity with hash chains, MACs, and related structures, and survey the limits of server-trusted designs [3], [4]. We apply standard HMAC-chain machinery per-session, with participant-facing verify exports and withdrawal semantics; our chain is deliberately scoped as tamper-evident, not tamper-proof.

Warning and rights comprehension. Warning research shows comprehension hinges on reception, attention, and terminology [1], that recipients frequently take no action on textual alerts [7], and that text characteristics measurably drive comprehension of rights-style warnings [2]. Banking Shield measures comprehension of *non-claims* —what the system does NOT do—as an inverted warning problem, in a formative setting only.

Data minimisation and design-based regulation. The GDPR’s data-minimisation and data-protection-by-design principles require systems to hold only what their purpose strictly needs and to hard-wire protection into architecture [5], [6], [8]. Our firewall is one concrete design-based mechanism, with the additional property that refusals produce evidence; we claim no compliance status under any regulation.

LLM guardrails and AI risk frameworks. A growing literature examines bias and risk across the life cycle of large language models [9] and catalogues AI trust, risk, and security management (TRiSM) controls, including runtime enforcement and output filtering [10]. Such approaches typically filter or monitor content emitted by a live model; Banking Shield instead makes the provider structurally unable to receive sensitive input, statically unable to egress, and schema-bound on output, with content scanning only as the final layer.

Open-banking consent UX. Open-banking customer-experience guidance treats consent and access dashboards as critical user-control surfaces [11], [12]. Banking Shield does not evaluate real open-banking consent efficacy; it evidences the integrity of a fictional consent-style flow.

Payee-confirmation services. Confirmation-of-payee and verification-of-payee schemes perform account-name matching

before payment initiation [13], [14]. Banking Shield’s payee scenario is fictional: the contribution is the metadata boundary, not name matching.

Per-response AI transparency artifacts. Model cards and datasheets document models or datasets as release-time artifacts [15], [16]. Banking Shield’s receipts are narrower: per-response, machine-checked records of what the explanation pipeline did and did not receive.

X. CONCLUSION

Banking Shield asks a narrow systems question with practical consequences: can a sensitive research prototype add an explanation layer while preserving machine-checkable boundaries around data collection, provider input, network egress, and reader-facing capability claims? The prototype’s answer is a structural pattern: fail-closed metadata firewalls, a deterministic offline provider behind provider-independent enforcement points, machine-checked claim discipline, and per-response receipts anchored in tamper-evident audit chains. At the evidence freeze, every automated gate passed. In a formative five-tester dry run, participants completed the lifecycle, the evidence pack contained zero sensitive values, and all testers understood the non-claim checklist.

The contribution is not banking protection and not live-LLM validation. It is a reproducible way to make absence claims inspectable: show what the system was allowed to construct, show what it rejected, show what the explanation provider received, and show what the output firewall would not let the system say. Future work should test the same contract with replay-logged live-provider integration, generated allowlist templates, externally countersigned receipts, independent security review, and a powered user study.

XI. LLM-ASSISTANCE DISCLOSURE

This paper was drafted with substantial assistance from a frontier language model operating under a logged, evidence-pack-constrained protocol: the model synthesised, critiqued, and drafted; it validated nothing. A second model-assisted independent-style review stage simulated five hostile reviewer roles and produced an author response before this preprint candidate. All factual statements trace to automated gate results, generated fixtures, frozen aggregate evidence, and explicit audit records. Citation candidates were verified against CrossRef metadata before inclusion; unresolved placeholders are not retained in this version.

REFERENCES

- [1] M. K. Lindell and R. W. Perry, “The protective action decision model: Theoretical modifications and additional evidence,” *Risk Analysis*, vol. 32, no. 4, pp. 616–632, 2012.
- [2] K. C. Scherr, S. J. Agauas, and J. Ashby, “The text matters: Eye movements reflect the cognitive processing of interrogation rights,” *Applied Cognitive Psychology*, vol. 30, no. 2, pp. 234–241, 2016.
- [3] T. R. Sree and S. M. S. Bhanu, “Secure logging scheme for forensic analysis in cloud,” *Concurrency and Computation: Practice and Experience*, vol. 31, no. 15, 2019.
- [4] A. Ali, A. Khan, M. Ahmed, and G. Jeon, “BCALS: Blockchain-based secure log management system for cloud computing,” *Transactions on Emerging Telecommunications Technologies*, vol. 33, no. 4, 2022.
- [5] A. Klein, R. Manini, and Y. Shi, “Across the pond: How US firms’ boards of directors adapted to the passage of the general data protection regulation,” *Contemporary Accounting Research*, vol. 39, no. 1, pp. 199–233, 2022.
- [6] K. Yeung and L. A. Bygrave, “Demystifying the modernized european data protection regime: Cross-disciplinary insights from legal and regulatory governance scholarship,” *Regulation & Governance*, vol. 16, no. 1, pp. 137–155, 2022.
- [7] G. Kim, A. Martel, D. Eisenman, M. Prelip, A. Arevian, K. L. Johnson, and D. Glik, “Wireless emergency alert messages: Influences on protective action behaviour,” *Journal of Contingencies and Crisis Management*, vol. 27, no. 4, pp. 374–386, 2019.
- [8] S. Bradshaw and L. DeNardis, “Privacy by infrastructure: The unresolved case of the domain name system,” *Policy & Internet*, vol. 11, no. 1, pp. 16–36, 2019.
- [9] J. Lee, Y. Hicke, R. Yu, C. Brooks, and R. F. Kizilcec, “The life cycle of large language models in education: A framework for understanding sources of bias,” *British Journal of Educational Technology*, vol. 55, no. 5, pp. 1982–2002, 2024.
- [10] P. P. Ray, “A review of TRISM frameworks in artificial intelligence systems: Fundamentals, taxonomy, use cases, key challenges and future directions,” *Expert Systems*, vol. 43, no. 3, 2026.
- [11] Open Banking Implementation Entity, “Consent & data sharing management,” Open Banking Customer Experience Guidelines, n.d., <https://standards.openbanking.org.uk/customer-experience-guidelines/introduction/consent-mgmt/latest/>.
- [12] World Bank, “The role of consumer consent in open banking,” n.d., <https://openknowledge.worldbank.org/entities/publication/5fa1c972-ea32-5504-a60f-42237b7385ac>.
- [13] Pay.UK, “Confirmation of payee,” n.d., <https://www.wearepay.uk/what-we-do/overlay-services/confirmation-of-payee/>.
- [14] European Payments Council, “Verification of payee,” n.d., <https://www.europeanpaymentscouncil.eu/what-we-do/other-schemes/verification-payee>.
- [15] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, “Model cards for model reporting,” in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019, pp. 220–229.
- [16] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.